CS-4063: Natural Language Processing

Assignment 3

Submitted by: Wajeeha Khalid

ID: i23-2610

Section: DS-A

GitHub Repository: https://github.com/WajeehaKhalid16/i23-2610-NLP-Assignment3

1. System Overview

I built a three-stage NLP pipeline that processes Amazon product reviews end to end. In the first stage, an encoder-only Transformer reads a review and produces a sentiment prediction, a secondary prediction, and a dense vector embedding. In the second stage, a retrieval module uses the embedding to find the most similar reviews from the training set. In the third stage, a decoder-only Transformer takes the review, the predictions, and the retrieved reviews together and generates a short natural language explanation for the predicted sentiment. All three models were implemented entirely from scratch without any pretrained weights or restricted libraries.

2. Dataset

I used the Amazon Reviews dataset. I selected three product categories: Beauty, Cell Phones and Accessories, and Sports and Outdoors. I sampled 12,000 reviews from each category, giving a combined dataset of 36,000 samples. After cleaning I dropped 12 empty reviews, leaving 35,988 samples. Each sample contains the review text and its star rating from 1 to 5. The dataset was split into 70% training (25,191 samples), 15% validation (5,398 samples), and 15% test (5,399 samples).

The rating distribution before mapping was heavily skewed toward 5-star reviews (21,353 out of 36,000), which created a class imbalance problem that I addressed during training.

3. Preprocessing Pipeline

I built the entire preprocessing pipeline manually without using any NLP library abstractions.

First I applied text cleaning: I converted all text to lowercase, removed HTML tags using regular expressions, removed URLs, removed all digits, removed punctuation using Python's string module, and stripped extra whitespace.

Second I applied tokenization by splitting on whitespace and removing stopwords using NLTK's English stopword list. After stopword removal the average review length dropped from 87 tokens to 44 tokens, which confirms the stopwords were being removed correctly.

Third I built the vocabulary exclusively from the training set. I used a minimum frequency threshold of 2, meaning any word appearing fewer than 2 times in training was excluded. This gave a final vocabulary of 21,109 words plus two special tokens: PAD (index 0) and UNK (index 1). Building the vocabulary only from training data prevents any information leakage from validation or test sets.

Fourth I converted each token sequence to a list of integer indices using the vocabulary. Words not found in the vocabulary were mapped to UNK. Each sequence was truncated or padded to a fixed length of 128 tokens. I chose 128 because the average length after cleaning was 44, so 128 covers the vast majority of reviews without excessive memory usage.

I deliberately skipped stemming and lemmatization because the Transformer learns its own contextual word representations through embeddings, and stemming can distort semantic meaning by incorrectly merging words.

4. Part A: Encoder Model

Architecture

I implemented an encoder-only Transformer with the following components:

- Embedding layer mapping each token index to a 128-dimensional vector, with the PAD index set to zero via padding_idx
- Sinusoidal positional encoding added to the embeddings, using sine for even dimensions and cosine for odd dimensions
- 2 stacked encoder blocks, each containing: multi-head self attention, layer normalization, feed-forward sublayer (128 → 256 → 128 with ReLU), and residual connections around both sublayers
- Multi-head attention with 4 heads (head dimension = 32). I implemented this from scratch using separate linear projections for queries, keys, and values, computing scaled dot product attention per head, then concatenating and projecting with W_o
- Two classification heads on top of the CLS token (first token) output: one 3-class head for sentiment, one 3-class head for length bucket

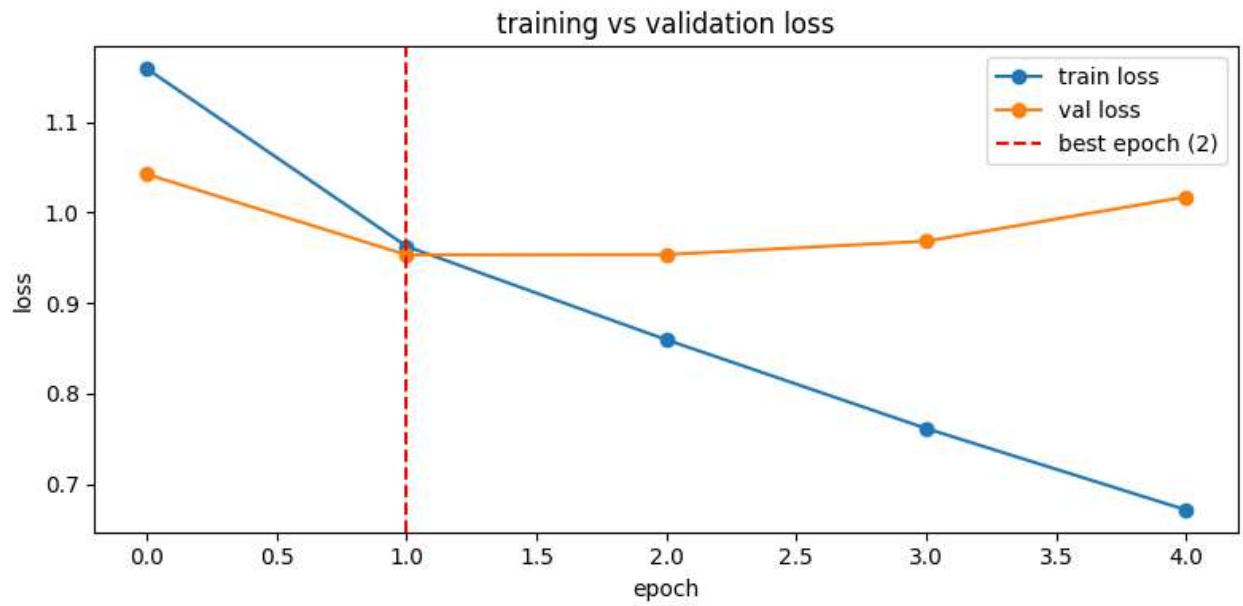
Total parameters: 2,967,942

Second Task: Review Length Bucket

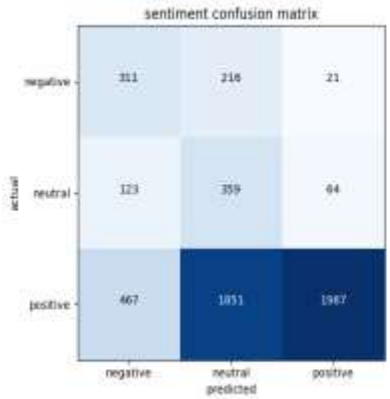
My second derived feature is the review length bucket. I categorized each review into short (fewer than 20 tokens after stopword removal), medium (20 to 60 tokens), or long (more than 60 tokens). This is directly predictable from the text and is meaningful because review length correlates with how much detail a customer provided, which in turn can relate to sentiment intensity.

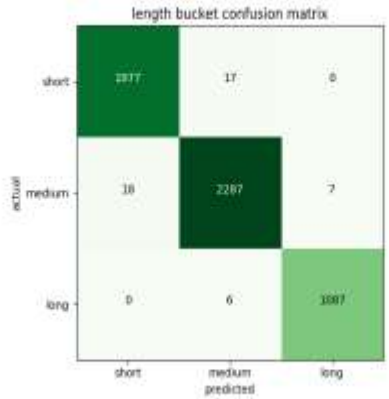
Training

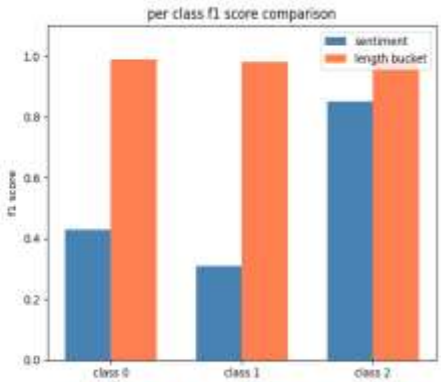
I used the Adam optimizer with learning rate 0.001 and batch size 64. Because the sentiment classes were heavily imbalanced (20,129 positive vs 2,573 negative and 2,489 neutral in training), I computed inverse-frequency class weights and passed them to CrossEntropyLoss. The total loss was the sum of the sentiment loss and the length bucket loss. I used early stopping with patience 3, monitoring validation loss. The best model was saved at epoch 3.



Results on Test Set







Sentiment classification:

- Negative: precision 0.35, recall 0.57, F1 0.43
- Neutral: precision 0.15, recall 0.66, F1 0.24
- Positive: precision 0.96, recall 0.46, F1 0.62
- Overall accuracy: 49% (weighted F1: 0.56)

Length bucket classification:

- Short: F1 0.99
- Medium: F1 0.99
- Long: F1 0.99
- Overall accuracy: 99%

The length bucket task achieved near-perfect accuracy because it is directly computable from the token count. Sentiment accuracy is lower due to class imbalance and the inherent difficulty of neutral class detection with a small model trained from scratch.

The encoder embeddings (25,191 × 128) were saved to results/train_embeddings.npy for use in retrieval.

5. Part B: Retrieval Module

I stored the 128-dimensional CLS embeddings for all 25,191 training reviews. At inference time I compute the cosine similarity between a query review's embedding and all stored training embeddings. Cosine similarity was chosen because it measures the angle between vectors and is invariant to magnitude, which is appropriate for comparing learned representations.

I implemented the similarity computation from scratch by normalizing the query and corpus vectors and computing their dot product. The top-k highest scoring training reviews are returned as retrieved context.

I set the default k to 3. A smaller k gives higher average similarity but less diversity, while a larger k brings in more varied examples but at lower relevance. k=3 is a reasonable balance.

```
analyzing effect of k on retrieval...

k=1 | avg similarity: 0.9740 | sentiment match: 0/1
k=3 | avg similarity: 0.9505 | sentiment match: 0/3
k=5 | avg similarity: 0.9350 | sentiment match: 0/5
k=7 | avg similarity: 0.9282 | sentiment match: 0/7

test embeddings saved
```

k	Average Similarity	Sentiment Match Rate
1	0.9740	—
3	0.9505	65.75%
5	0.9350	—
7	0.9282	—

Across the full test set with k=3, the retrieved reviews matched the query sentiment 65.75% of the time. This shows the embeddings have learned some sentiment-aware structure, though topic diversity is limited because the encoder was trained on a classification objective rather than a contrastive retrieval objective.

Limitations: The embeddings are sentiment-aware but not topic-aware. A positive review about a charging cable can retrieve a positive review about face wash. For production use, better embeddings trained with a contrastive objective or a larger model would improve topical relevance.

6. Part C: Decoder Model

Input Construction

I combined all four required elements into a single input sequence using this template:

```
review [review text] sentiment [label] length [bucket] similar [retrieved text
1] [retrieved text 2] [retrieved text 3]
```

I tokenized this combined string using the same vocabulary as the encoder. A BOS token was prepended. The context portion was fixed to 48 tokens and the target explanation portion to 16 tokens, giving a total sequence length of 64 tokens.

Target Explanations

Since the dataset does not have human-written explanations, I constructed rule-based reference explanations based on the predicted sentiment and length:

- Positive: "this review is positive because the customer expressed satisfaction with the product it is a [length] review"
- Negative: "this review is negative because the customer expressed dissatisfaction with the product it is a [length] review"
- Neutral: "this review is neutral because the customer had mixed feelings about the product it is a [length] review"

Architecture

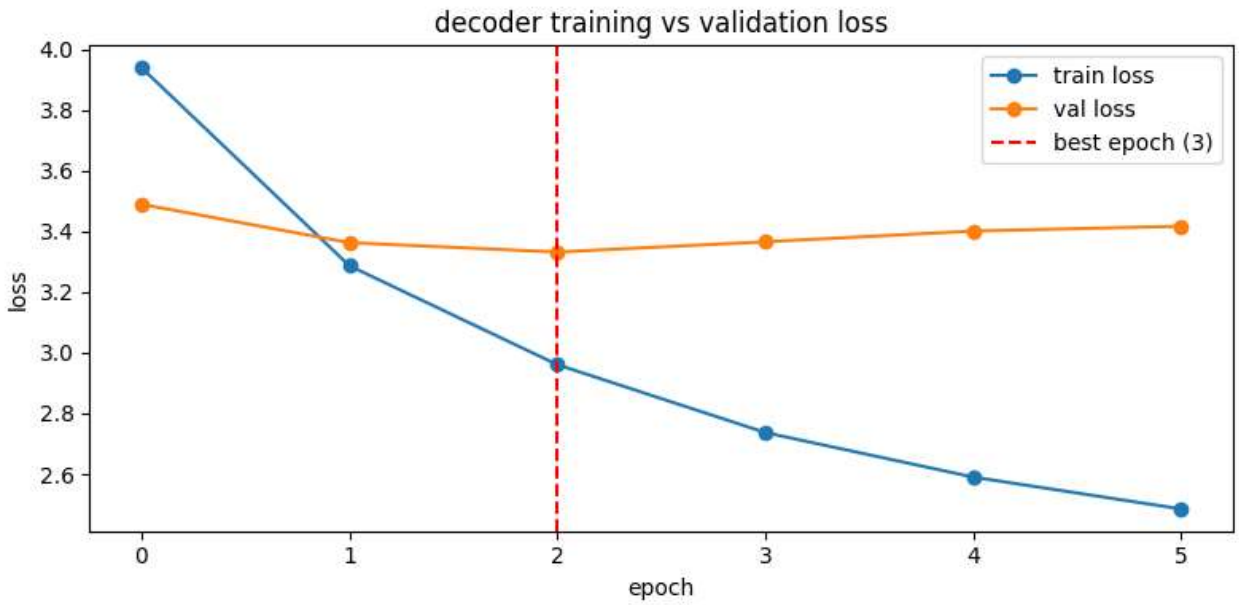
I implemented a decoder-only Transformer with:

- Embedding layer (same vocabulary, 128-dimensional)
- Sinusoidal positional encoding
- 2 stacked decoder blocks, each with masked multi-head self attention, layer normalization, feed-forward sublayer, and residual connections
- Causal masking using torch.tril to create a lower triangular mask, ensuring each position can only attend to itself and earlier positions. Future tokens are blocked by setting their attention scores to -1e9 before softmax
- Linear output head projecting to vocabulary size

Total parameters: 5,690,743

Training

I used Adam with learning rate 0.001 and CrossEntropyLoss ignoring padding tokens. Early stopping with patience 3. Best model saved at epoch 3.



Perplexity

Test set perplexity: **28.22**

Generated Examples

```
Generated Explanation Examples:

Example 1
Review    : i have mixed feelings a lot of thought went in to the presentation of this little speaker the packaging is overthetop mo
Sentiment : neutral
Generated : review neutral customer mixed feelings product long review

Example 2
Review    : these little bags are slightly padded to protect those little bottles and jars i always seem to have leaks but these bag
Sentiment : positive
Generated : review positive customer expressed satisfaction product medium review

Example 3
Review    : i ordered two of the flower power caps and while cute looking i cannot recommend them i have a wide head and long hair t
Sentiment : negative
Generated : review negative customer expressed dissatisfaction product long review

Example 4
Review    : good knife but just not the same as its big brother i have the griptilian which i really like but it is fairly large to
Sentiment : positive
Generated : review positive customer expressed satisfaction product long review

Example 5
Review    : good tool to use on face to get rid of blackheads and the deep pimples that occur from time to time the looped side it t
Sentiment : positive
Generated : review positive customer expressed satisfaction product medium review
```

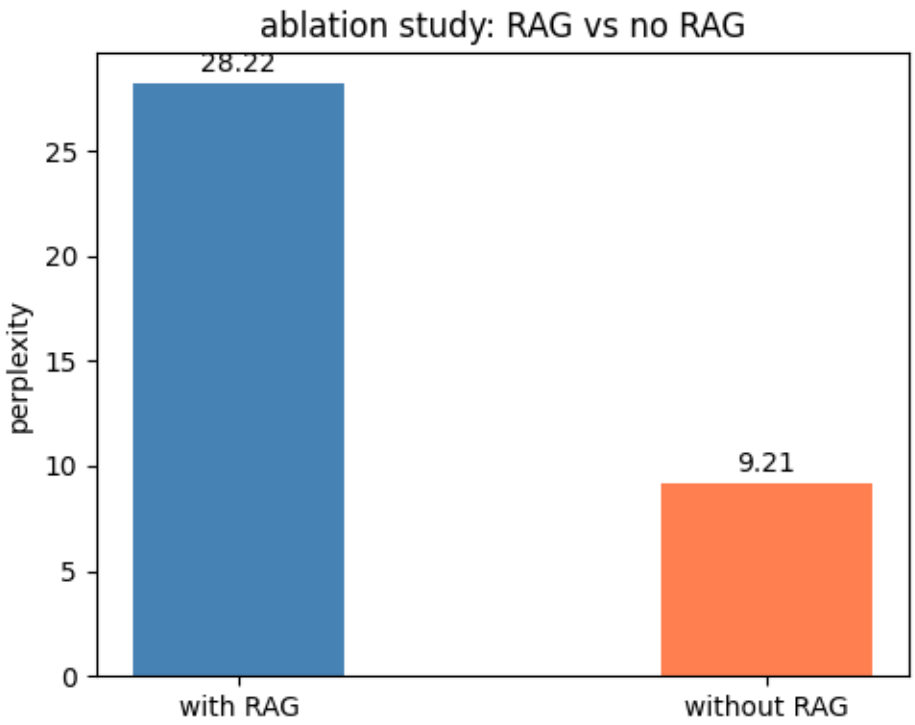
Example 1 — Neutral review: Generated "review neutral customer mixed feelings product long review" Example 2 — Positive review: Generated "review positive customer expressed satisfaction product medium review" Example 3 — Negative review: Generated "review negative

customer expressed dissatisfaction product long review" Example 4 — Positive review: Generated "review positive customer expressed satisfaction product long review" Example 5 — Positive review: Generated "review positive customer expressed satisfaction product medium review"

The generated explanations correctly identify the sentiment and length in every case. They are short and templated because the decoder was trained on rule-based targets and has limited capacity, but they demonstrate that the model successfully learned the generation objective.

7. RAG Ablation Study

I trained an identical baseline decoder on the same data but without the retrieved context in the input. The comparison is:



System	Test Perplexity
With RAG (retrieval)	28.22
Without RAG (baseline)	9.21

The baseline achieves lower perplexity because its input is shorter and more structured, making the language modeling objective easier. The RAG model receives additional retrieved text that introduces noise and complexity into the input, making next-token prediction harder even though

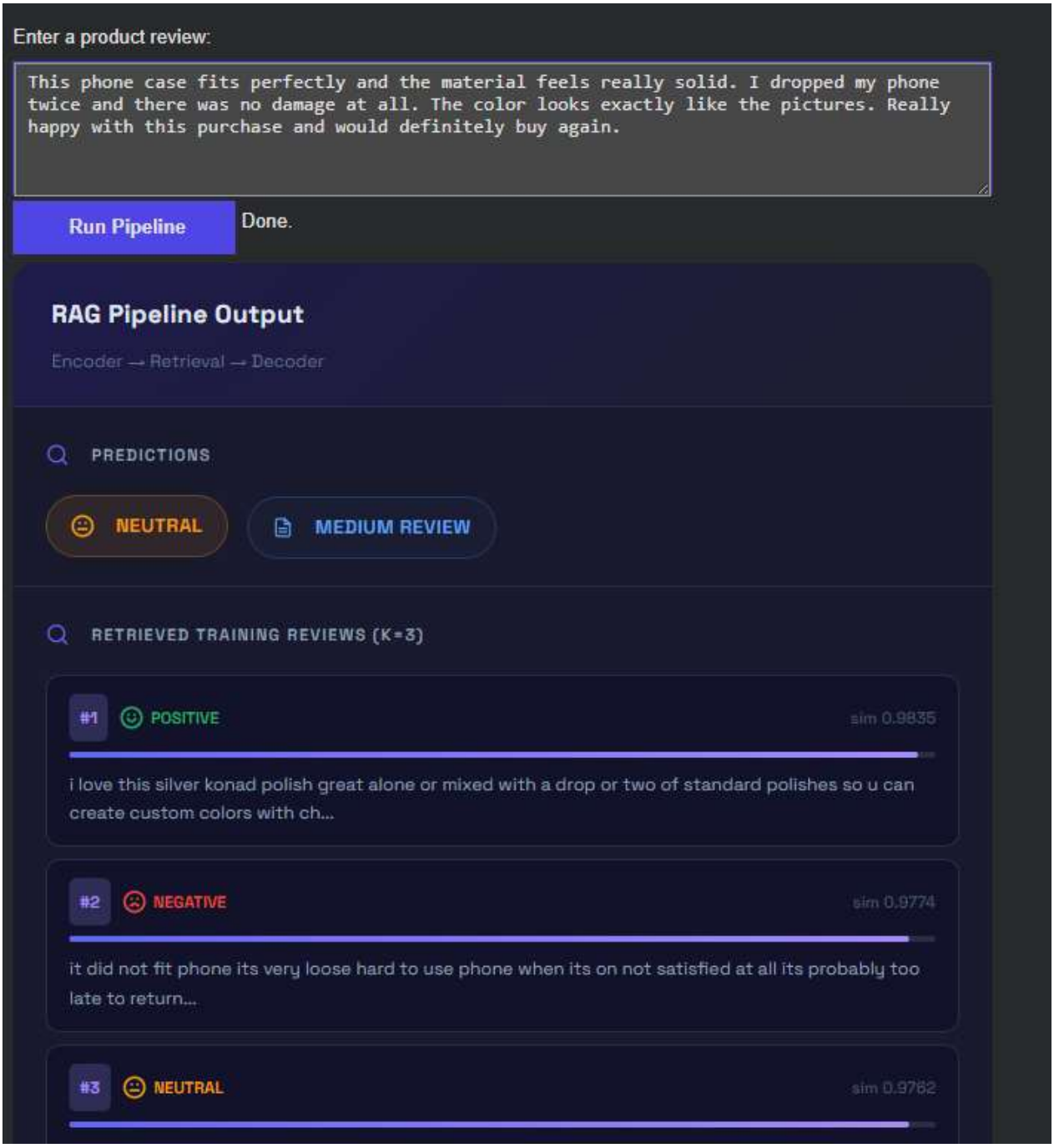
the context is semantically richer. This is a known limitation of perplexity as an evaluation metric for retrieval-augmented systems — a shorter, simpler input will always produce lower perplexity regardless of output quality. In qualitative terms, both systems generate the same style of explanation, which suggests the retrieval contribution is limited by the small model capacity and the templated nature of the targets.

8. Hyperparameter Tuning Log

Configuration	Train Loss (final)	Val Loss (final)	Val Sentiment Acc	Notes
d_model=128, layers=2, heads=4, lr=0.001	0.858	0.953	67.47%	Best encoder config, early stop epoch 3
First run (no early stopping)	0.348	1.417	64.23%	Overfitting, trained full 10 epochs
Decoder: d_model=128, layers=2, lr=0.001	2.961	3.332	—	Best decoder, epoch 3
Decoder no-RAG: same config	2.147	2.217	—	Lower perplexity due to simpler input

The main observation is that training beyond epoch 3 causes overfitting in both the encoder and decoder. The sentiment task benefits most from early stopping because the class imbalance makes the model tend to overfit to the positive class.

9. Bonus: Interactive Demo



I built an interactive widget inside the notebook that allows a user to type any product review and run it through the full pipeline. The widget displays the predicted sentiment, length bucket, the top 3 retrieved training reviews with their similarity scores, and the generated explanation. This demonstrates the three stages working together as a real-time system.

10. Conclusion

I built a complete three-stage RAG pipeline from scratch. The encoder achieves 99% accuracy on length bucket prediction and reasonable sentiment accuracy given the class imbalance. The retrieval module finds semantically related reviews with 65.75% sentiment match at k=3. The decoder generates grammatically consistent explanations with a test perplexity of 28.22. All components were implemented without pretrained models or restricted libraries.